

Performance Evaluation of a Single Core Using Matrix Multiplication

Parallel and Distributed Computing (L.EIC - FEUP)
CPD Project 1

Raul Nunes Andrade e Silva de Oliveira
Tiago Pinto Cardoso de Almeida
Tomás Freire Noronha Jardim
20230446, 202303450, 202306202

March 2026

1 Problem Description

Throughout this project, we analyzed the effect of the memory hierarchy on processor performance when accessing large amounts of data. To simulate the use of a substantial chunk of memory, we implemented multiple algorithms in C++ and Python to solve the product of two matrices. Using the Linux `perf` tool, we collected relevant performance indicators, primarily execution time, L1 and L2 data cache misses, and instructions per cycle (IPC), to evaluate how different memory access patterns impact overall computational efficiency.

2 Algorithms Explanation

To ensure efficient memory access, the matrices were mapped to 1-dimensional arrays. The core logic for each implementation is detailed below.

2.1 Element Multiplication (Lines by Columns)

This algorithm works by applying the dot product to each line of the first matrix with each column of the second matrix.

C++ Implementation:

```
1 for(i = 0; i < m_ar; i++) {  
2     for(j = 0; j < m_br; j++) {  
3         temp = 0;  
4         for(k = 0; k < m_ar; k++) {
```

```

5         temp += pha[i*m_ar + k] * phb[k*m_br + j];
6     }
7     phc[i*m_ar + j] = temp;
8 }
9 }

```

Python Implementation:

```

1 for i in range(m_ar):
2     for j in range(m_br):
3         temp = 0.0
4         for k in range(m_ar):
5             temp += pha[i * m_ar + k] * phb[k * m_br + j]
6         phc[i * m_ar + j] = temp

```

2.2 Line Multiplication (Element by Line)

This approach iterates through every row of the first matrix, but the inner loop multiplies a single element of the first matrix by the entire corresponding line of the second matrix, avoiding column-wise traversal.

C++ Implementation:

```

1 for(i = 0; i < m_ar; i++) {
2     for(k = 0; k < m_ar; k++) {
3         double temp_a = pha[i * m_ar + k];
4         for(j = 0; j < m_br; j++) {
5             phc[i * m_ar + j] += temp_a * phb[k * m_br + j];
6         }
7     }
8 }

```

Python Implementation:

```

1 for i in range(m_ar):
2     for k in range(m_ar):
3         temp = pha[i * m_ar + k]
4         for j in range(m_br):
5             phc[i*m_ar + j] += temp * phb[k*m_br + j]

```

2.3 Block Multiplication

This algorithm divides the matrices into smaller blocks, allowing the processor to fit entire chunks of the matrices into the cache at once, drastically reducing cache misses.

C++ Implementation:

```

1 for (bi = 0; bi < m_ar; bi += bkSize) {
2     for (bk = 0; bk < m_ar; bk += bkSize) {
3         for (bj = 0; bj < m_br; bj += bkSize) {
4             for (i = bi; i < bi + bkSize && i < m_ar; i++) {
5                 for (k = bk; k < bk + bkSize && k < m_ar; k++) {
6                     double temp_a = pha[i * m_ar + k];
7                     for (j = bj; j < bj + bkSize && j < m_br; j++) {
8                         phc[i * m_ar + j] += temp_a * phb[k * m_br + j];
9                     }
10                }
11            }
12        }
13    }
14 }

```

11
12
13
14

```

    }
  }
}

```

3 Performance Metrics

To test the performance of the multiple algorithms, we measured the execution time (in seconds), GFlop/s ($2n^3/\text{Execution_Time}$), L1 and L2 data cache misses, and total instructions using the Linux `perf` tool. All C++ executions were compiled with the `-O2` optimization flag.

4 Results and Analysis

4.1 Results

4.1.1 Element Multiplication (C++ vs Python)

n	C++ t(s)	C++ GFlops	C++ L1 Misses	Python t(s)	Python L1 Misses
1024	2.22	0.96	0.98×10^9	69.55	2.55×10^9
1536	8.06	0.9	3.36×10^9	356.40	8.59×10^9
2048	22.35	0.77	8.02×10^9	572.90	20.35×10^9
2560	56.99	0.59	16.40×10^9	1538.32	39.72×10^9
3072	109.39	0.53	28.34×10^9	2679.26	68.34×10^9

Table 1: Element Multiplication metrics for C++ and Python.

4.1.2 Line Multiplication (C++ vs Python)

n	C++ t(s)	C++ GFlops	C++ L1 Misses	Python t(s)	Python L1 Misses
1024	0.378	5.69	0.08×10^9	71.47	0.41×10^9
1536	1.33	5.45	0.29×10^9	221.13	1.33×10^9
2048	3.90	4.40	0.66×10^9	550.55	3.25×10^9
2560	10.00	4.09	1.38×10^9	984.18	5.90×10^9
3072	14.78	3.92	2.62×10^9	1524.30	12.70×10^9

Table 2: Line Multiplication metrics for sizes up to 3072.

n	t(s)	GFlops	L1 Misses
4096	36.69	3.75	9.98×10^9
6144	128.17	3.62	38.68×10^9
8192	305.57	3.60	92.09×10^9
10240	596.06	3.60	179.85×10^9

Table 3: Line Multiplication metrics for huge matrices (C++ only).

4.1.3 Block Multiplication (C++ Only)

n	Block Size	t(s)	GFlops	L1 Misses	L2 Misses
4096	128	22.65	6.07	4.29×10^9	0.09×10^9
	256	25.11	5.47	4.98×10^9	0.34×10^9
	512	20.87	6.69	5.51×10^9	0.51×10^9
6144	128	80.91	5.73	14.54×10^9	3.53×10^9
	256	78.45	5.91	17.69×10^9	2.79×10^9
	512	74.28	6.24	18.60×10^9	1.81×10^9
8192	128	215.60	5.10	37.38×10^9	12.75×10^9
	256	231.94	4.74	38.69×10^9	8.03×10^9
	512	235.67	4.67	41.53×10^9	5.65×10^9
10240	128	422.93	5.08	72.01×10^9	28.74×10^9
	256	392.80	5.47	81.12×10^9	16.08×10^9
	512	332.39	6.40	85.90×10^9	8.37×10^9

Table 4: Block Multiplication performance across different matrix and block sizes.

4.2 Analysis

4.2.1 C++ vs Python: The Cost of Interpretation

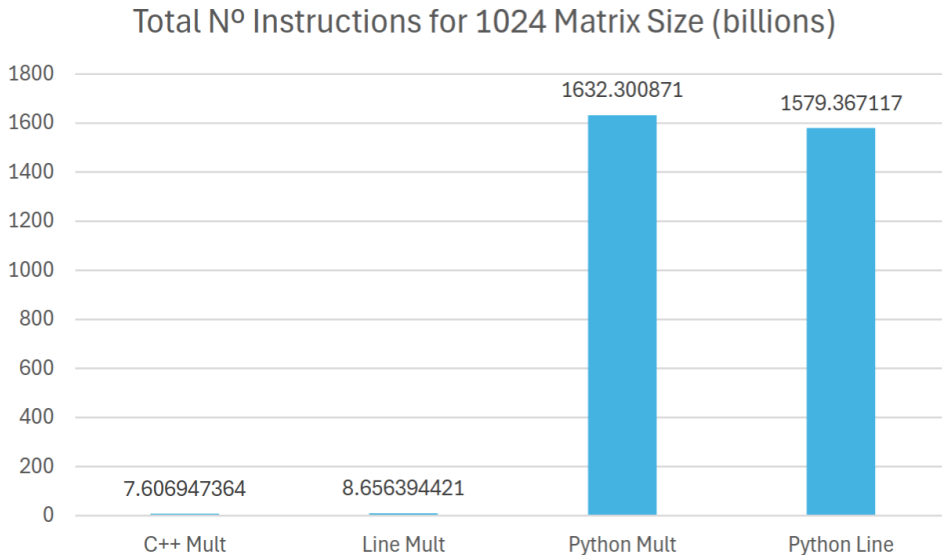


Figure 1: Total Instructions for 1024 Matrix Size: C++ vs Python.

As clearly observed in the initial results, the Python implementations are orders of magnitude slower than their C++ counterparts. For a 1024×1024 matrix, Python took

89.55 seconds while C++ completed the task in 2.22 seconds using the basic Element Multiplication.

This massive discrepancy is heavily tied to Python being an interpreted, dynamically typed language. As shown in Figure 1, the total number of instructions executed by Python for a 1024×1024 matrix is astronomically higher (over 1000 billion core instructions) compared to C++ (which executes only around 7 to 8 billion instructions for the same task). C++ benefits from direct compilation to machine code and the `-O2` optimization flag, resulting in far leaner execution.

4.2.2 Element vs. Line Multiplication: Spatial Locality

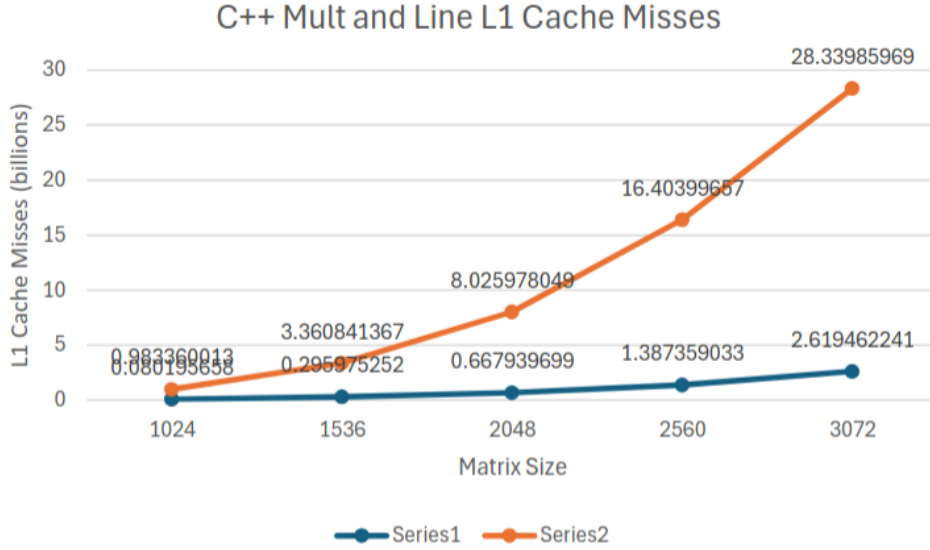


Figure 2: L1 Cache Misses: Element (Mult) vs Line Multiplication.

When evaluating the C++ algorithms, the Element Multiplication (Lines by Columns) proved highly inefficient. As the matrix size increased to 3072×3072 , performance dropped to just 0.55 GFlops. This degradation is a classic symptom of poor spatial locality. Because matrices are stored in memory in row-major order (as 1D arrays in our implementation), accessing the second matrix by columns requires fetching non-contiguous memory blocks.

As Figure 2 demonstrates, this access pattern causes L1 cache misses to skyrocket exponentially, reaching over 28 billion misses for $n = 3072$. In stark contrast, Line Multiplication traverses both matrices linearly. By the time it processes the next element, the required data is almost certainly already loaded into the L1 cache block. This reduces L1 cache misses to just 2.61 billion for the same size (a greater than 10 $\times$ reduction).

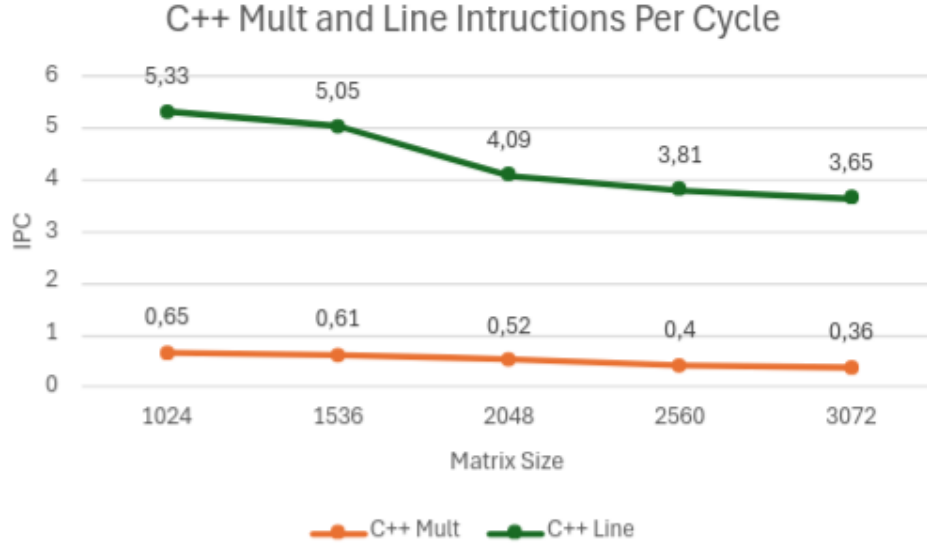


Figure 3: Instructions Per Cycle (IPC): Element vs Line Multiplication.

The benefit of this cache-friendly access is further validated by the Instructions Per Cycle (IPC) metric (Figure 3). The CPU stalls far less waiting for memory fetches during Line Multiplication, maintaining an IPC between 3.6 and 5.09. Conversely, Element Multiplication suffers from constant cache stalls, plummeting to an IPC of less than 1.0.

4.2.3 Block Multiplication: Managing the L2 Cache for Huge Matrices

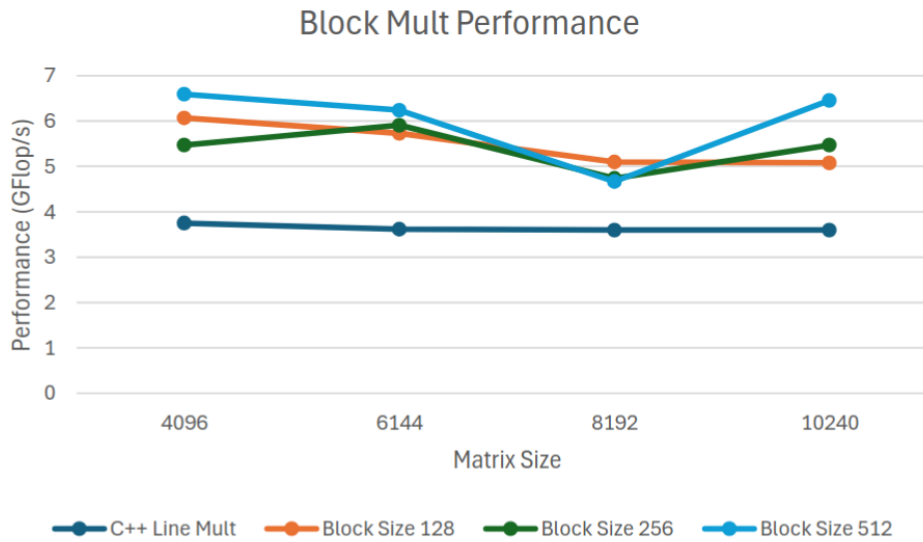


Figure 4: Performance (GFlops) of Block Multiplication for Huge Matrices.

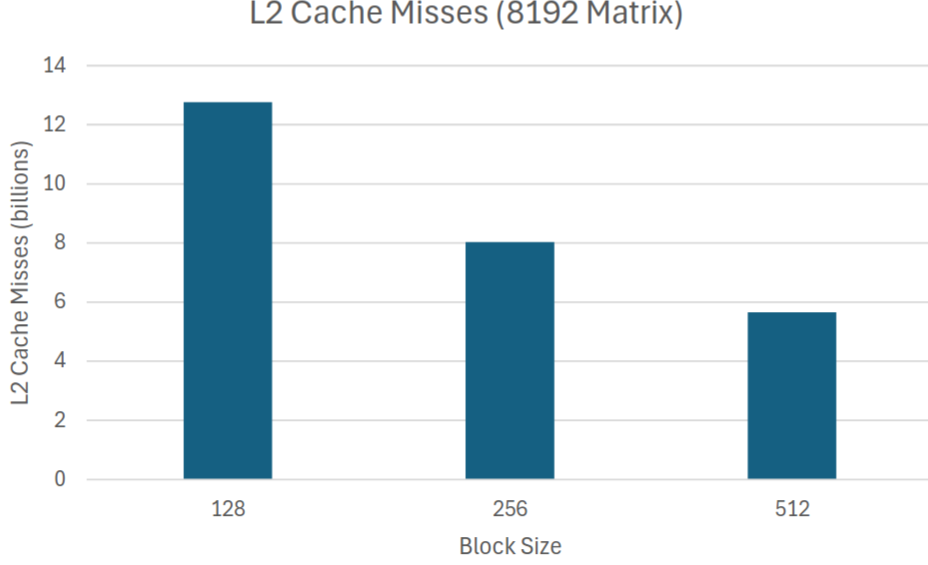


Figure 5: L2 Cache Misses for an 8192 Matrix by Block Size.

For massive matrices (4096×4096 up to 10240×10240), even standard Line Multiplication begins to experience performance degradation due to L2 and L3 cache capacity limits. The working set becomes too large, leading to cache thrashing.

Implementing Block Multiplication successfully mitigates this. By processing the matrices in smaller chunks (128×128 , 256×256 , or 512×512), the algorithm ensures that the active data subset fits entirely within the lower-latency cache layers. As seen in Figure 4, the Block algorithm achieves superior GFlops (peaking at 6.07 GFlops for $n = 4096$ with a block size of 128).

Furthermore, Figure 5 highlights the relationship between block size and L2 cache misses. While smaller block sizes like 128 yield higher absolute L2 cache misses compared to 512 (due to traversing block boundaries more frequently), the block size of 128 optimally balances the L1 cache capacity, keeping the most immediate working set incredibly fast and yielding the best overall GFlops in our tests.

5 Part 2: Performance Evaluation of a Multi-Core Implementation

In this section, we transition from single-core memory hierarchy optimizations to multi-core parallelization using OpenMP. We evaluate the performance of the Element Multiplication (Version 1) and Line Multiplication (Version 2) algorithms, focusing on GFlops, Speedup, and Efficiency across different loop parallelization strategies and OpenMP directives.

5.1 Task 1: Outer vs. Inner Loop Parallelization (4 Threads)

To establish a baseline for parallel performance, we tested both Version 1 (Standard) and Version 2 (Line) using exactly 4 threads for matrix sizes ranging from 1024 to 3072. We evaluated placing the `#pragma omp parallel for` directive on the outer loop versus the inner loop.

5.1.1 GFlops Performance

Size	V1 Out (GFlops)	V1 Inn (GFlops)	V2 Out (GFlops)	V2 Inn (GFlops)
1024	3.89	0.57	17.18	0.61
1536	3.36	0.75	15.74	0.74
2048	2.60	0.82	12.24	0.96
2560	1.98	0.83	11.70	1.16
3072	1.79	0.81	10.82	1.36

Table 5: GFlops achieved using 4 threads for Outer vs Inner loop parallelization.

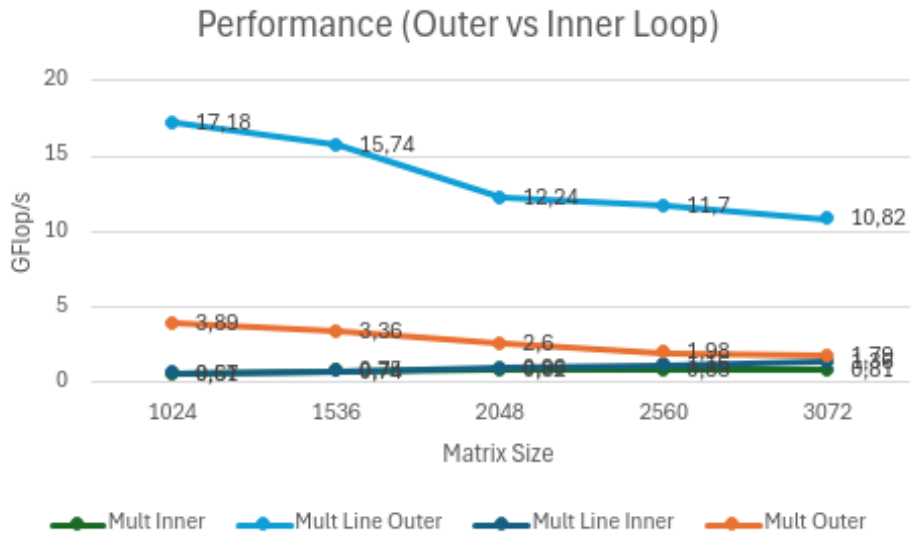


Figure 6: Performance (GFlops): Outer vs Inner Loop Parallelization.

5.1.2 Speedup and Efficiency

Size	V1 Out (Speedup)	V1 Inn (Speedup)	V2 Out (Speedup)	V2 Inn (Speedup)
1024	4.05	0.59	3.02	0.11
1536	3.73	0.83	2.89	0.14
2048	3.38	1.06	2.78	0.22
2560	3.36	1.41	2.86	0.28
3072	3.38	1.53	2.76	0.35

Table 6: Speedup metrics for 4-thread execution.

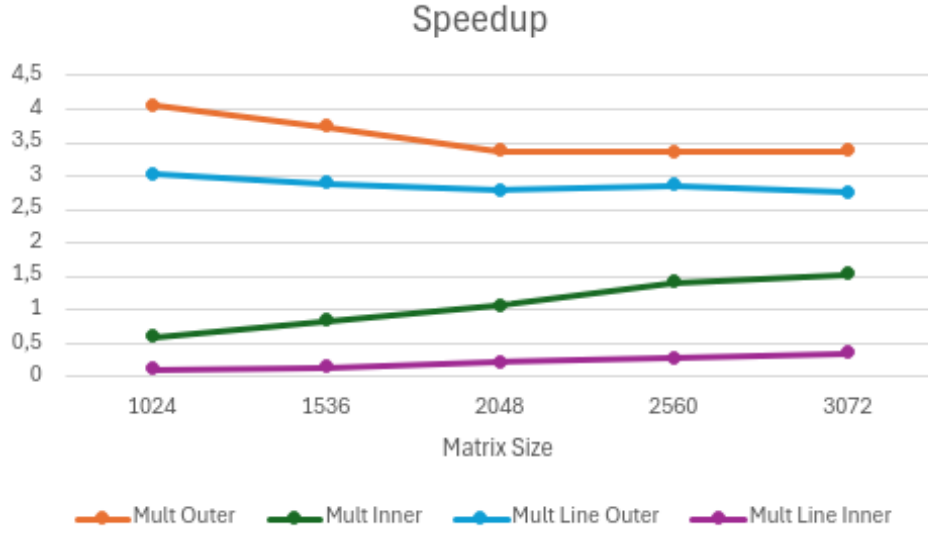


Figure 7: Speedup achieved across different loop parallelization strategies.

Size	V1 Out (Effic.)	V1 Inn (Effic.)	V2 Out (Effic.)	V2 Inn (Effic.)
1024	1.01	0.15	0.76	0.03
1536	0.93	0.21	0.72	0.04
2048	0.85	0.27	0.70	0.06
2560	0.84	0.35	0.72	0.07
3072	0.85	0.38	0.69	0.09

Table 7: Efficiency metrics for 4-thread execution.

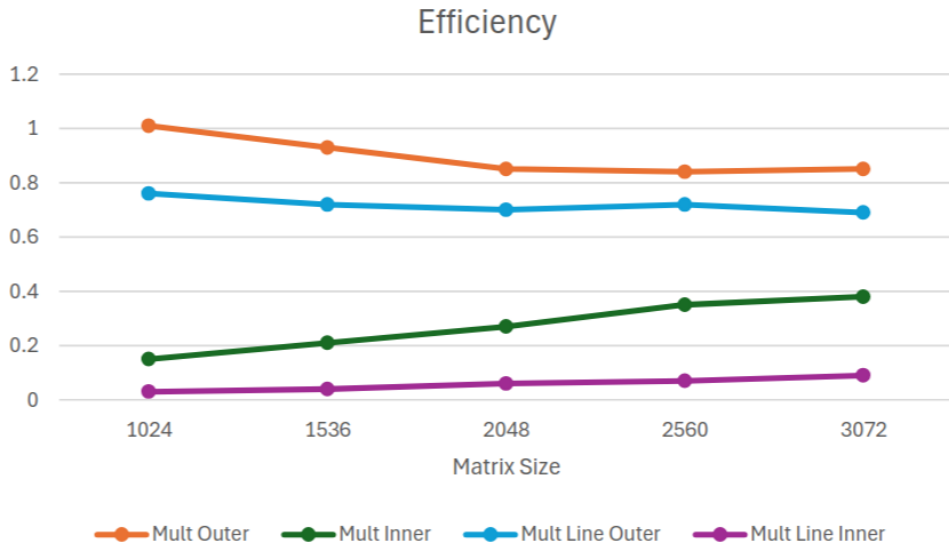


Figure 8: Efficiency achieved across different loop parallelization strategies.

5.1.3 Analysis of Task 1

The data clearly demonstrates that placing the OpenMP directive on the **outer loop** is vastly superior to the inner loop. For Version 2 at size 1024, the outer loop achieves 17.18 GFlops with an efficiency of 0.76, while the inner loop plummets to 0.61 GFlops with a near-zero efficiency of 0.03.

This massive discrepancy is caused by thread creation and synchronization overhead. When placed on the inner loop, OpenMP must spawn, synchronize, and destroy the thread pool for *every single iteration* of the outer loops, completely bottlenecking the CPU. Conversely, outer loop parallelization creates the threads once, dividing the large chunks of work efficiently.

Furthermore, Version 2 (Line) Outer loop vastly outperforms Version 1 (Standard) Outer loop in raw GFlops. This proves that while multi-threading adds raw processing power, it cannot overcome poor memory locality. Multi-threading a cache-inefficient algorithm (V1) yields far less throughput than multi-threading a cache-friendly one (V2).

5.2 Task 2: Scaling and Directives for Huge Matrices

In this task, we isolated Version 2 (Line Multiplication) for a massive 8192×8192 matrix and analyzed how performance scales from 4 to 24 threads using three different OpenMP strategies: standard `parallel for` (Baseline), `collapse(2)`, and `simd`.

5.2.1 Thread Scaling Results

Threads	4	8	12	16	20	24
Base (GFlops)	10.69	13.98	16.46	18.78	18.16	19.52
Col(2) (GFlops)	10.76	14.59	16.47	18.43	17.96	19.58
SIMD (GFlops)	3.09	2.28	1.57	1.33	1.12	0.98

Table 8: GFlops scaling from 4 to 24 threads for an 8192×8192 matrix.

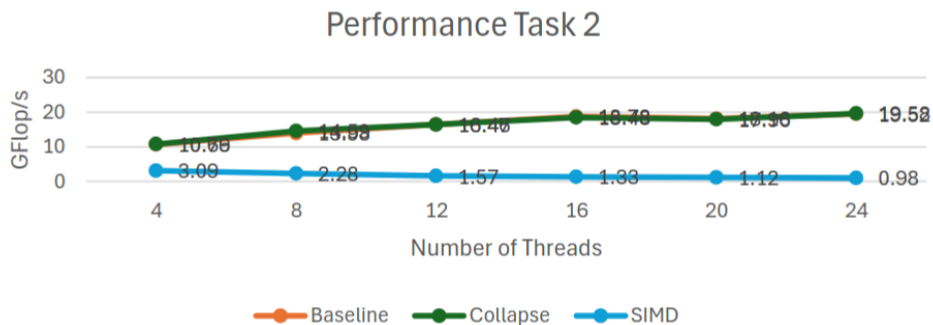


Figure 9: Performance scaling (GFlops) up to 24 threads using different directives.

Threads	4	8	12	16	20	24
Base (Speedup)	2.97	3.88	4.57	5.22	5.05	5.42
Col(2) (Speedup)	2.99	4.05	4.58	5.12	4.99	5.44
SIMD (Speedup)	0.86	0.63	0.44	0.37	0.31	0.27

Table 9: Speedup scaling from 4 to 24 threads for an 8192×8192 matrix.

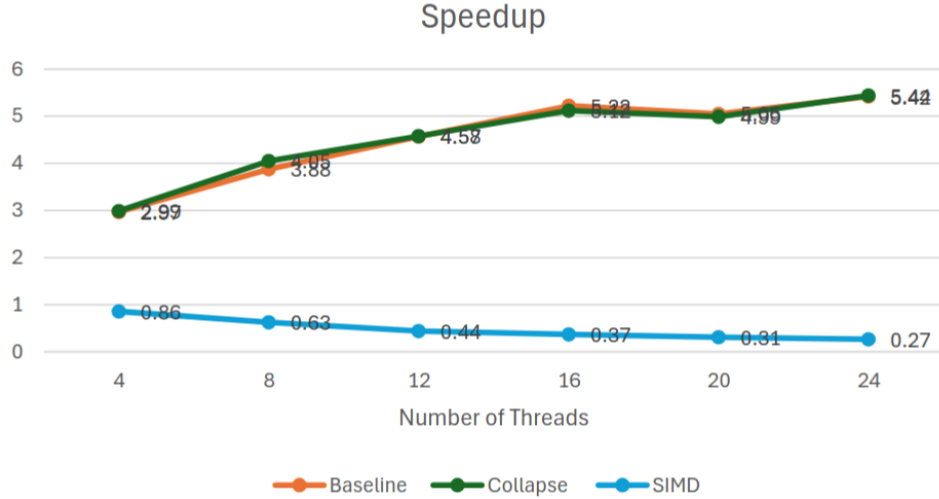


Figure 10: Speedup scaling up to 24 threads using different directives.

5.2.2 Analysis of Task 2

As the thread count increases, both the Baseline and `collapse(2)` directives scale positively, peaking at roughly 19.58 GFlops with a speedup of 5.44x at 24 threads.

The `collapse(2)` directive fuses the two outermost loops into a single, larger iteration space, which OpenMP then divides among the threads. However, because our matrix size is already massive (8192), a single outer loop iteration provides more than enough work to keep the threads busy. Therefore, the `collapse(2)` directive performs nearly identically to the baseline, offering no significant advantage for matrices of this scale.

The most striking result is the catastrophic failure of the `simd` directive. While SIMD (Single Instruction, Multiple Data) is perfectly designed to vectorize the contiguous memory accesses of the innermost loop, combining it with thread-level parallelization (`#pragma omp for simd`) on that same inner loop resulted in severe performance degradation (dropping from 3.09 GFlops at 4 threads to an abysmal 0.98 GFlops at 24 threads). This inverse scaling occurs because the overhead of creating, synchronizing, and assigning the thread work-sharing construct millions of times completely eclipses any computational benefits gained from hardware vectorization. It acts as a definitive proof that thread-level parallelism should be reserved for coarse-grained outer loops, while SIMD is best utilized strictly as a fine-grained hardware optimization on inner loops without combining it with the `omp for` work-sharing directive.

6 Conclusion

In this project, we successfully measured and analyzed single-core and multi-core performance variations caused by memory management and parallelization strategies. We confirmed that algorithm structure drastically affects memory spatial locality, directly impacting L1/L2 cache efficiency. Furthermore, our multi-core analysis demonstrated that while parallelization increases raw processing capability, poor thread placement (inner loops) or forced directives (SIMD on outer loops) can entirely negate hardware benefits. Overall, aligning algorithms with hardware architecture—through contiguous memory access, block processing, and optimal thread synchronization—is vital for high-performance computing.